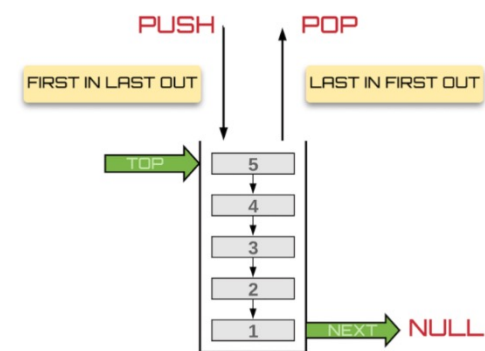
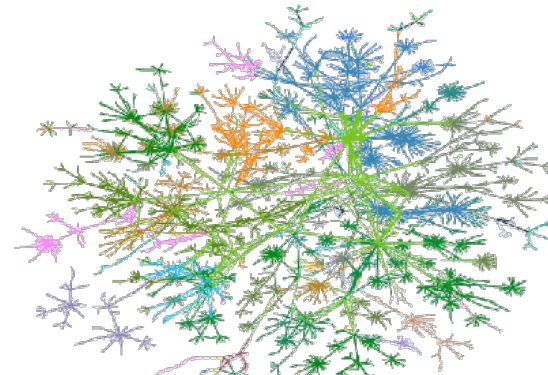
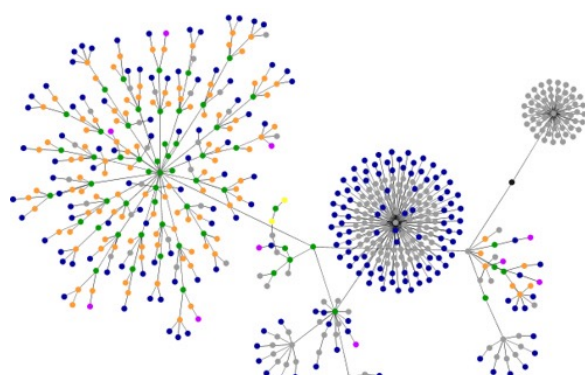
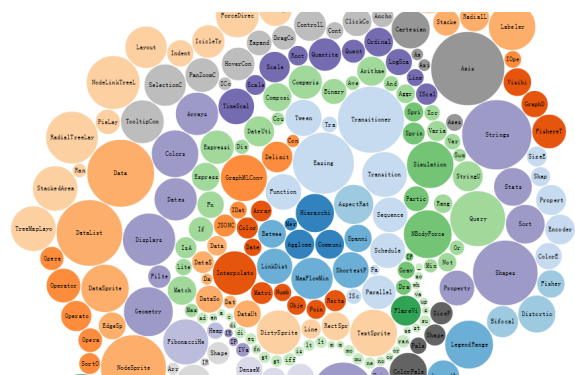


《计算机科学导论》课程

算法 · 图探索

上海交通大学计算机科学导论课程组



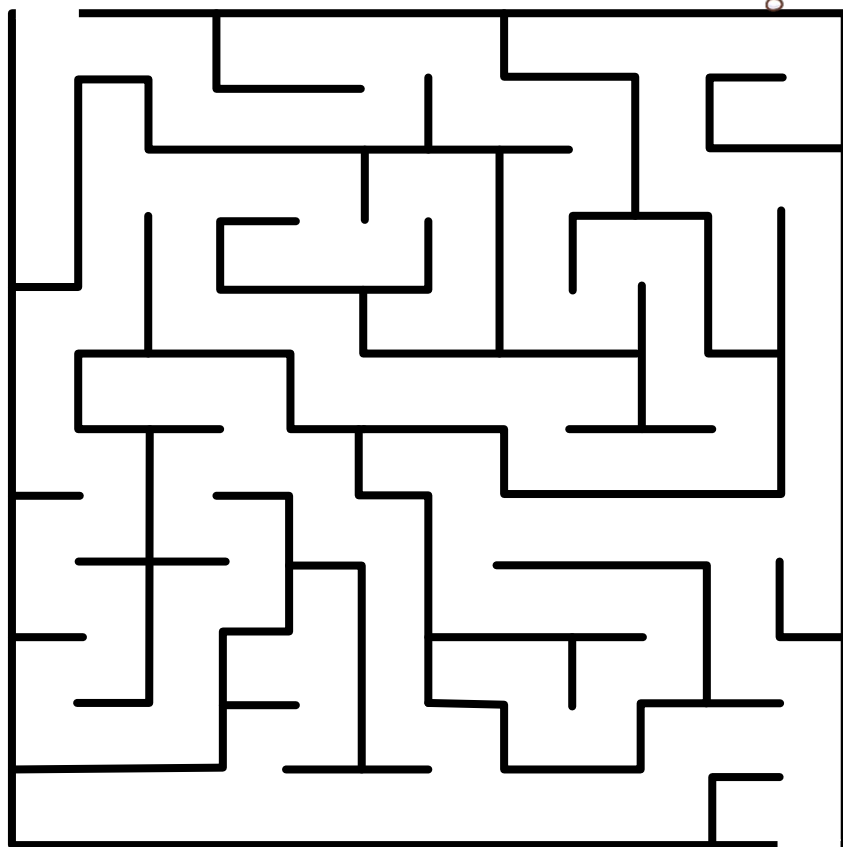
致谢：本课件由高晓汎设计，刘伟@ParisTech2019M.SJTU辅助编写，高晓汎终校定稿。

生活中的搜索问题

进入迷宫如何自动寻找出口



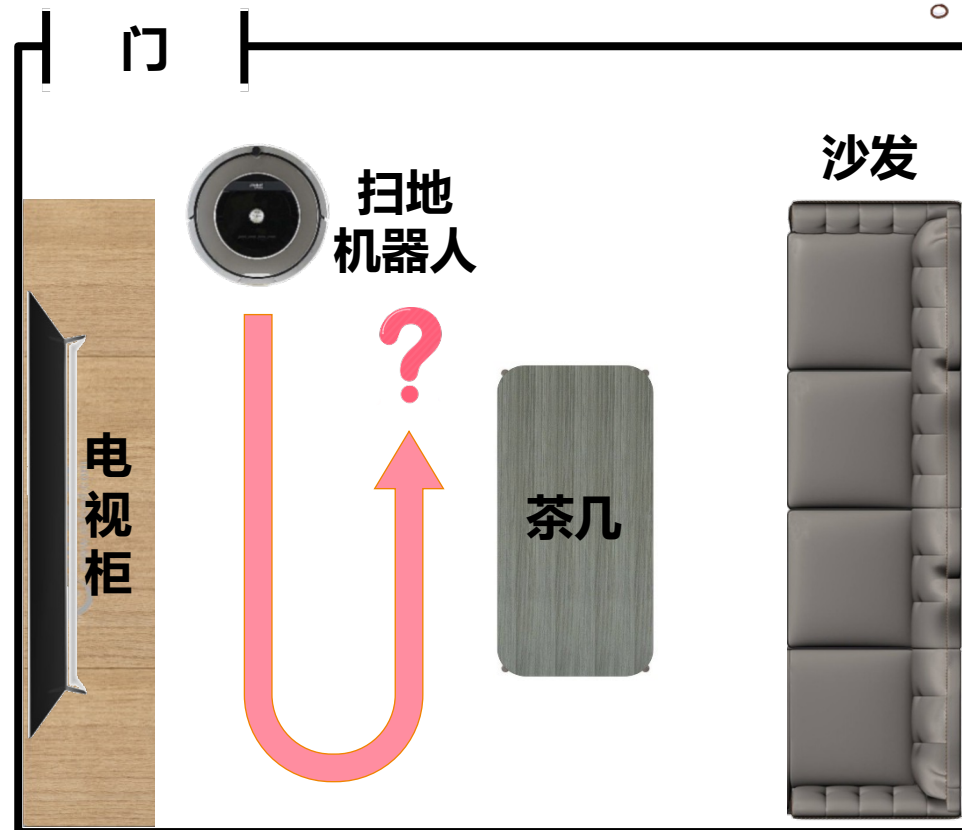
入口



出口

寻找满足条件的路线

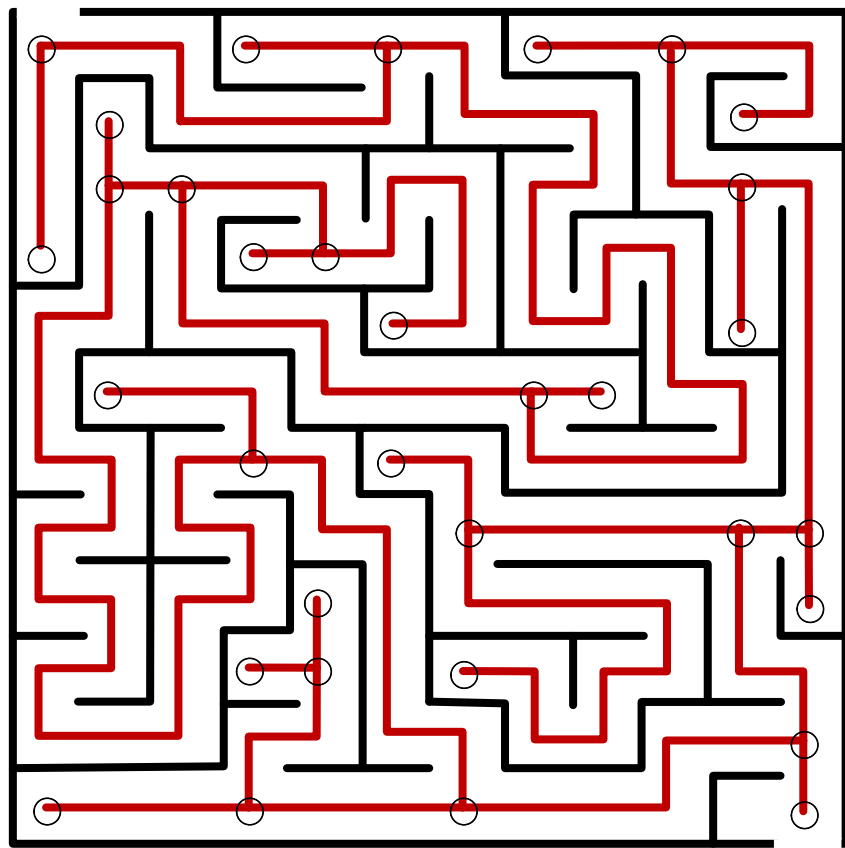
扫地机器人如何自动规划行走路径



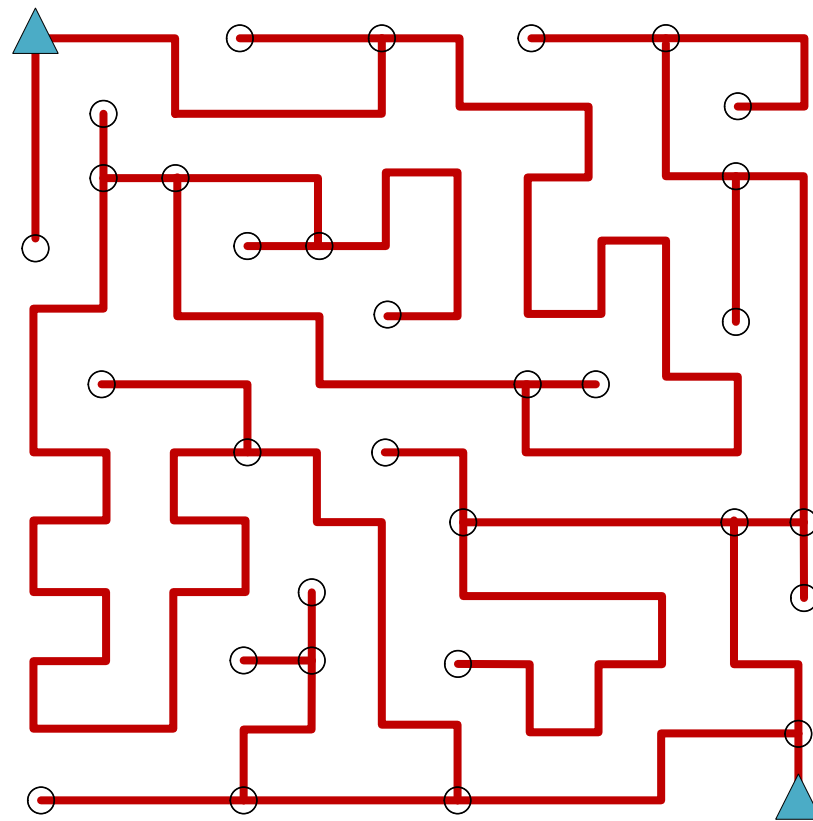
探寻所有可行的空间

迷宫搜索

进入迷宫后寻找出口

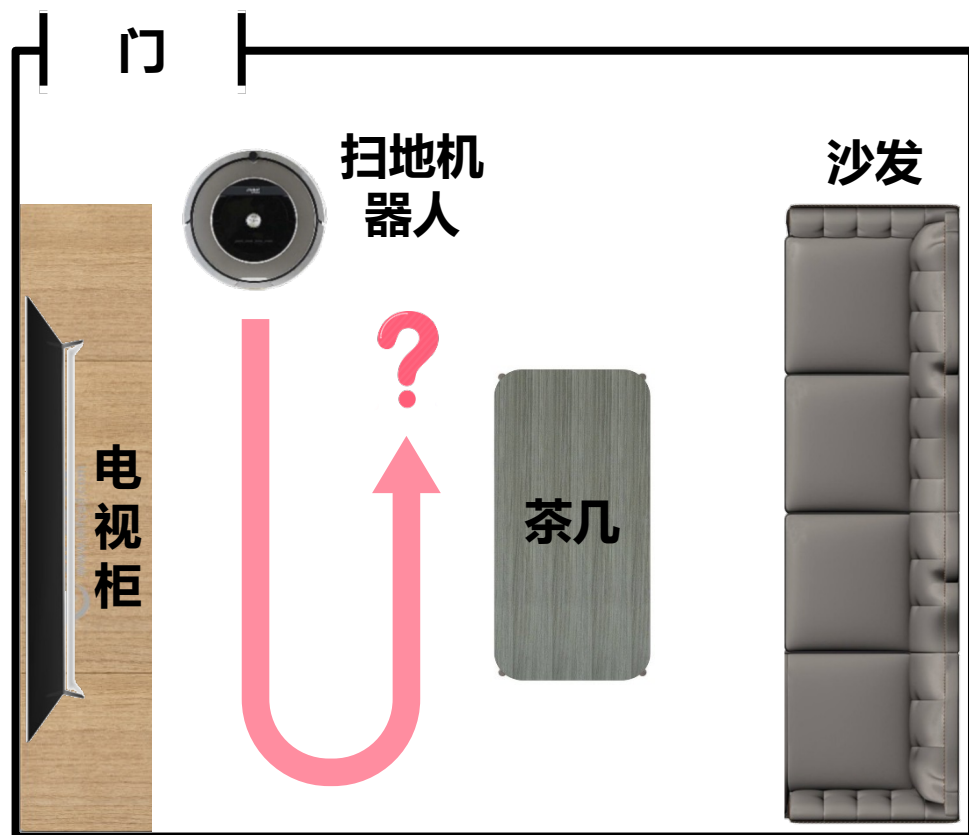


1. 画出迷宫中所有可能的路线
2. 将路线上每个分支处或终止处作为**顶点**
3. 将两个顶点之间的路线作为一条**边**

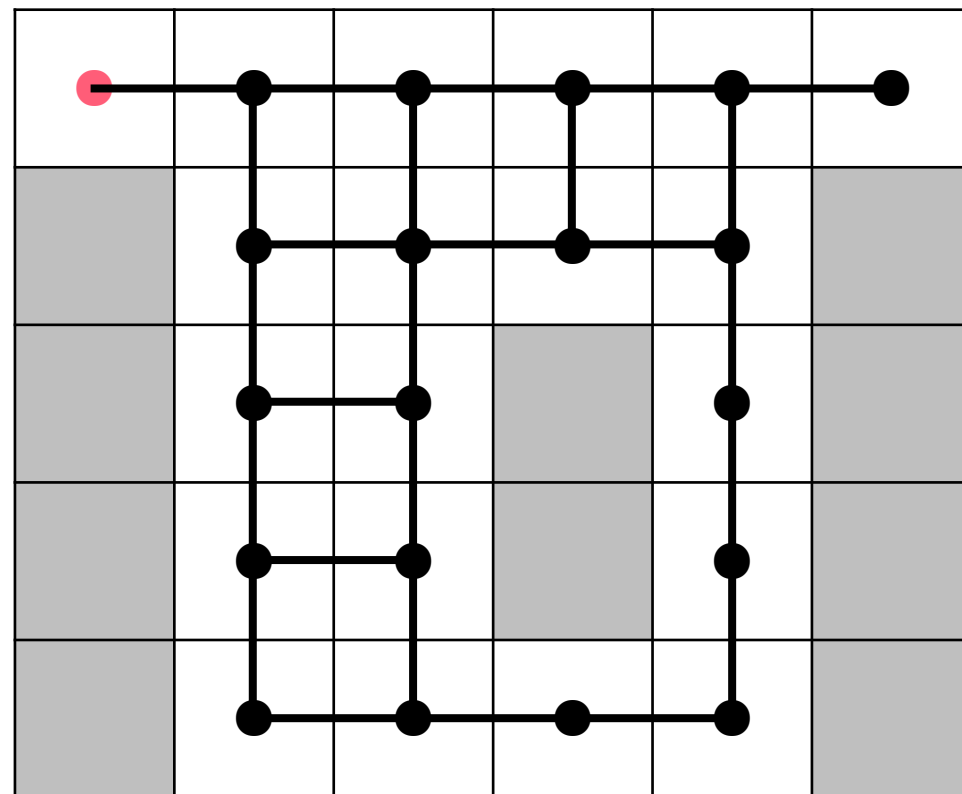


扫地机器人

扫地机器人打扫房间

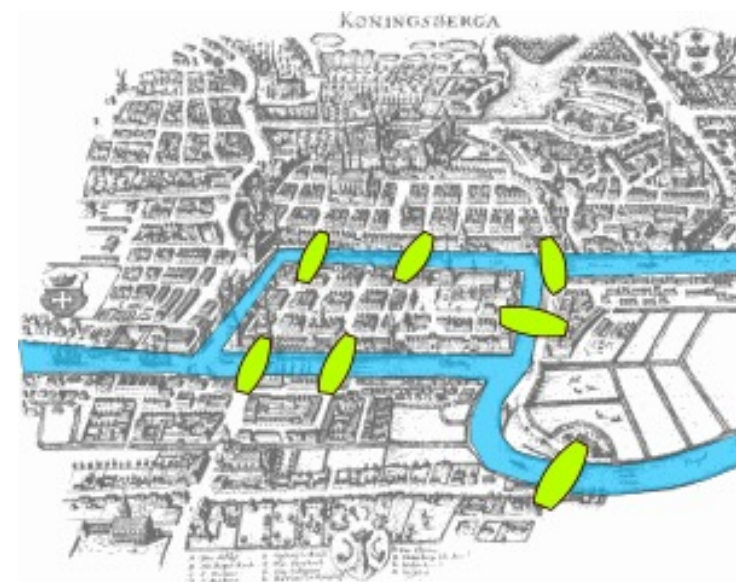


1. 以扫地机器人大小栅格化房间
2. 以每个格子中心为顶点连成图



图论：哥尼斯堡七桥问题

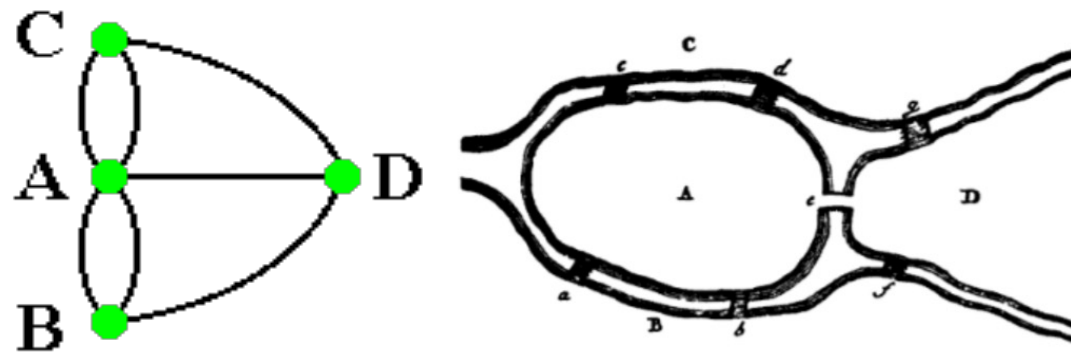
普雷格尔河(Pregel)从哥尼斯堡镇(Konigsberg, Prussia-now Kaliningrad Russia)中穿过，而河中有两个小岛，小岛与河岸间由7座桥彼此连接。于是有游客提出问题：能否从河岸或小岛出发，通过每一座桥，而且仅仅通过一次，最后回到原地。



图论：欧拉解

瑞士数学家欧拉(Leonhard Euler)在1736年发表论文“**哥尼斯堡的七座桥**”，对这个问题给出了否定的回答。他将河岸和小岛作为图的顶点，七座桥为边，构成一个无向图，则问题化为图论中的问题。

这是用图论方法建立的第一个模型。**(图论的元年)**



The Seven Bridges of Königsberg

T

1.

he branch of geometry that deals with magnitudes has been zealously studied throughout the past, but there is another branch that has been almost unknown up to now; Leibniz spoke of it first, calling it the "geometry of position" (*geometria situs*). This branch of geometry deals with relations dependent on position alone, and investigates the properties of position; it does not take magnitudes into consideration, nor does it involve calculation with quantities. But as yet no satisfactory definition has been given of the problems that belong to this geometry of position or of the method to be used in solving them. Recently there was announced a problem that, while it certainly seemed to belong to geometry, was nevertheless so designed that it did not call for the determination of a magnitude, nor could it be solved by quantitative calculation; consequently I did not hesitate to assign it to the geometry of position, especially since the solution required only the consideration of position, calculation being of no use. In this paper I shall give an account of the method that I discovered for solving this type of problem, which may serve as an example of the geometry of position.

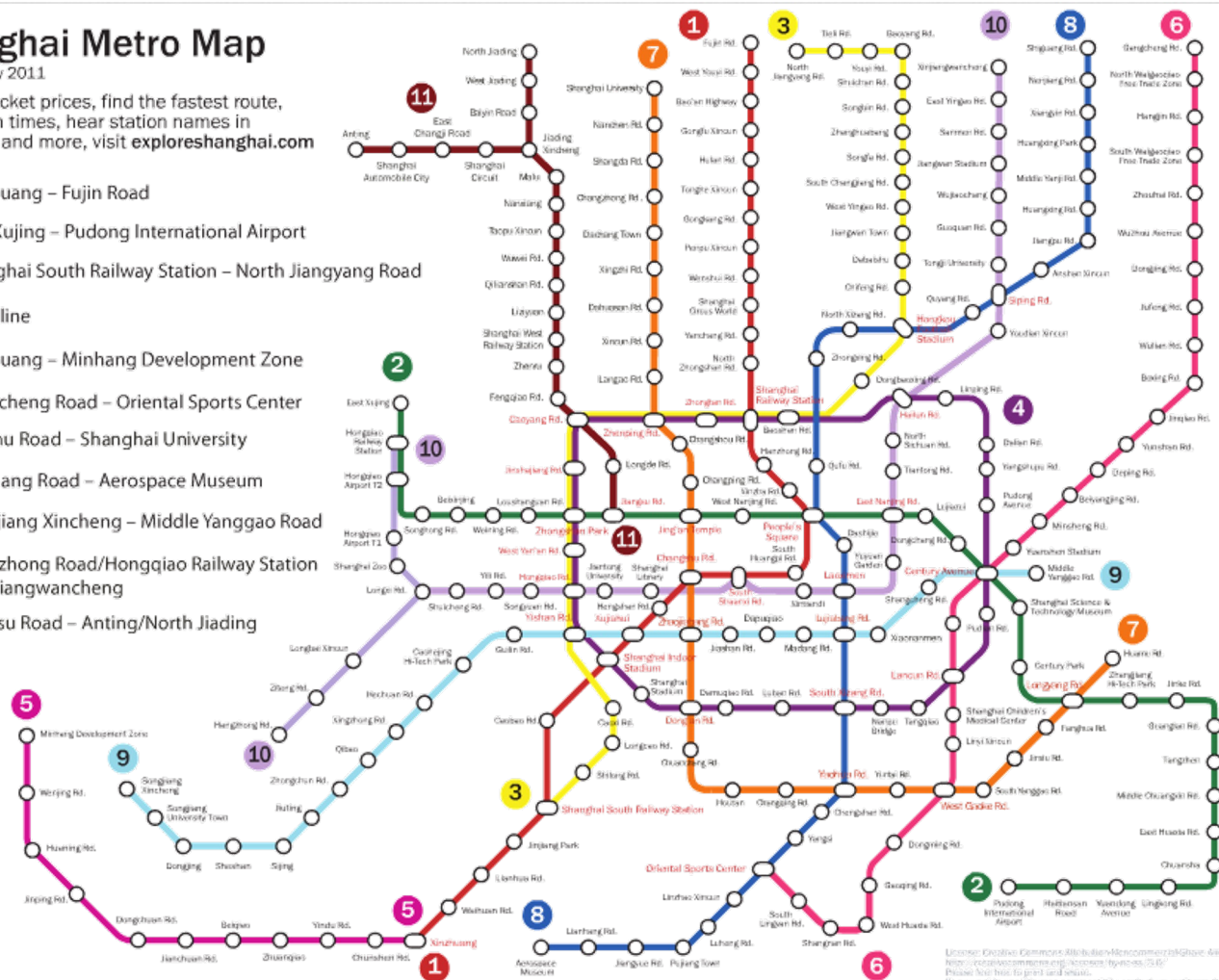
图论：图的示例

Shanghai Metro Map

Updated May 2011.

To check ticket prices, find the fastest route, check train times, hear station names in Mandarin, and more, visit exploreshanghai.com

- 1 Xinzhuang – Fujin Road
- 2 East Xujing – Pudong International Airport
- 3 Shanghai South Railway Station – North Jiangyang Road
- 4 Loop line
- 5 Xinzhuang – Minhang Development Zone
- 6 Gangcheng Road – Oriental Sports Center
- 7 Huamu Road – Shanghai University
- 8 Shiguang Road – Aerospace Museum
- 9 Songjiang Xincheng – Middle Yanggao Road
- 10 Hangzhong Road/Hongqiao Railway Station – Xinjiangwancheng
- 11 Jiangsu Road – Anting/North Jiading

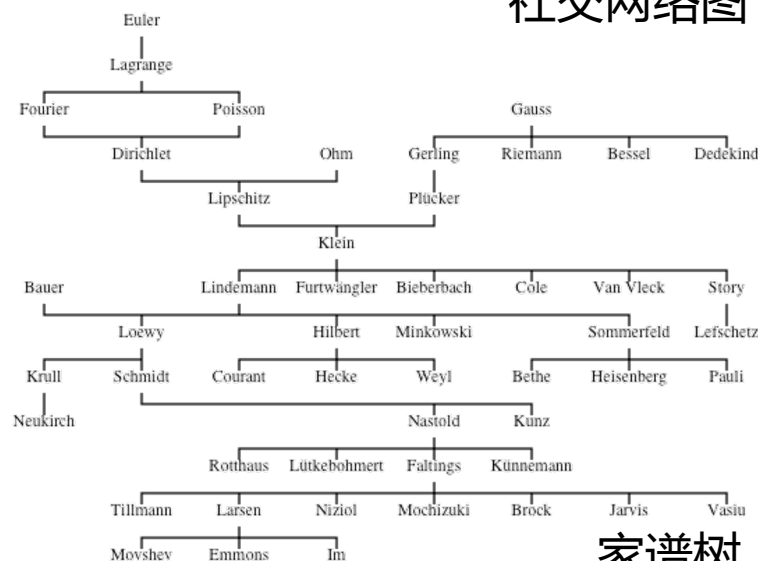


上海地铁交通图



Social networks

社交网络图



家谱树

图论：发展与研究对象

图论发展

- 18、19世纪的数学家仅将图论视为一般的数学游戏，并未及时获得足够的发展。
- 1936年，匈牙利数学家柯尼希(Konig)出版图论第一本专著《有限图与无限图理论》，总结了图论200年的成果。
- 20世纪中后期，随着计算机领域的开拓，图论得到长足发展，现已成长为数学科学一个重要的独立学科。
- 分支：算法图论、极值图论、网络图论、代数图论、随机图论、模糊图论、超图论...

研究对象

- 世界由事物组成，事物之间有联系。图可以直观地描述事物及其间联系。可见，图模型几乎可用于任何领域。
- 图论(graph theory)就是以这种结点和边构成的图为研究对象。

图论是简朴而机敏的学科 —— 王树禾



图论基本概念

深度优先搜索

广度优先搜索

01.图论基本概念：无向图

【定义】 称二元组 $G = (V, E)$ 是一个**无向图**(Undirected graph), 若

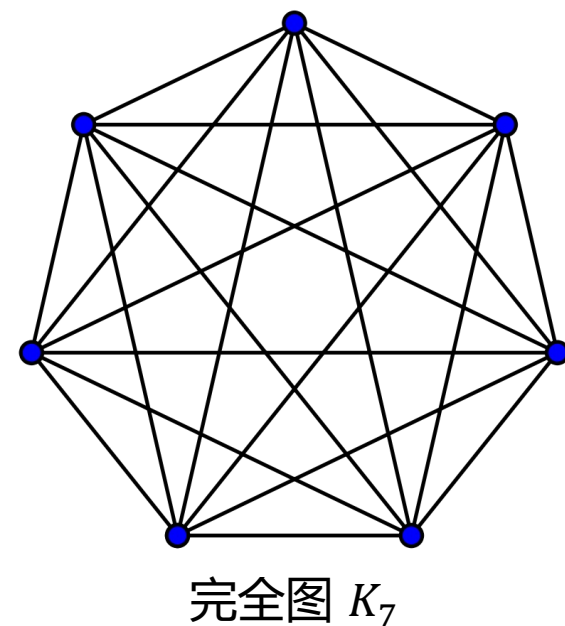
- V 是一个非空有限集合, 称为顶点集, $V = \{v_1, v_2, \dots, v_n\}$;
- E 是 V 中元素的无序对所组成的集合, 称为边集, $E = \{e_1, e_2, \dots, e_m\}$

e_k 可记为无序的顶点对 (v_i, v_j)

- 称 e_k 与 v_i, v_j **关联**; e_k 连接 v_i 和 v_j ;
- v_i, v_j 是 e_k 的**端点**; 称 v_i 和 v_j **相邻** (adjacent或neighbors).

与顶点 v 相关联的边数称为 v 的**度** (degree), 记为 $d(v)$

完全图 (complete graph): 任意两结点间都有边的简单图。 n 个顶点的完全图记作 K_n



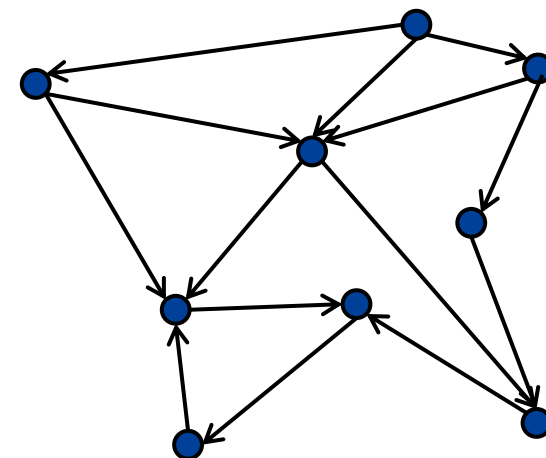
01.图论基本概念：有向图

【定义】 设 G 是有向图， $e = (u, v)$ 是 E 中元素，则称 u 为 e 的起点或尾， e 为 u 的出边，并称 v 为 e 的终点或头， e 为 v 的入边。

若对任意的 $u, v \in V$ ，均有 $(u, v) \in E$ 和 $(v, u) \in E$ ，则称 G 为有向完全图。

【定义】 在有向图中，顶点 v 的出边数称为 v 的出度，记作 $d^+(v)$ 。顶点 v 的入边数称为 v 的入度，记作 $d^-(v)$ 。

顶点 v 的关联边数称作 v 的度，记为 $d(v)$ ，即

$$d(v) = d^+(v) + d^-(v)$$


01.图论基本概念：子图

令 $V(G)$ 表示图 G 的顶点集， $E(G)$ 表示图 G 的边集。

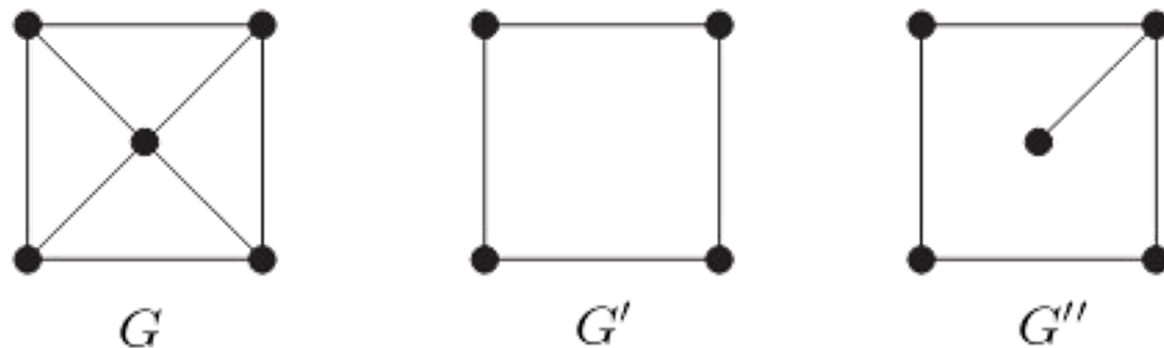
若 $V(H) \subseteq V(G)$ ， $E(H) \subseteq E(G)$ ，则称图 H 是图 G 的**子图(subgraph)**，记作 $H \subseteq G$ 。若 $H \subseteq G$ ，且 $H \neq G$ ，则称 H 是 G 的**真子图**，记作 $H \subsetneq G$ 或 $H \subset G$ 。

若 $H \subseteq G$ ， $V(H) = V(G)$ ，则称 H 是 G 的**生成子图**或**支撑子图(spanning subgraph)**。

如果 $V(H) \subseteq V(G)$ ，且 H 包含了 G 在节点集 $V(H)$ 中的所有边，则称 H 是 G 的**导出子图(induced subgraph)**。

例：

- G' 和 G'' 都是 G 的子图
- G' 是 G 的导出子图，而 G'' 不是
- G'' 是 G 的生成子图，而 G' 不是



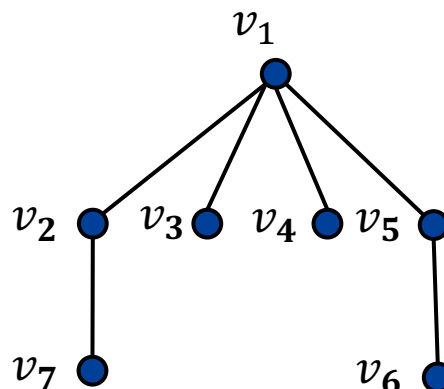
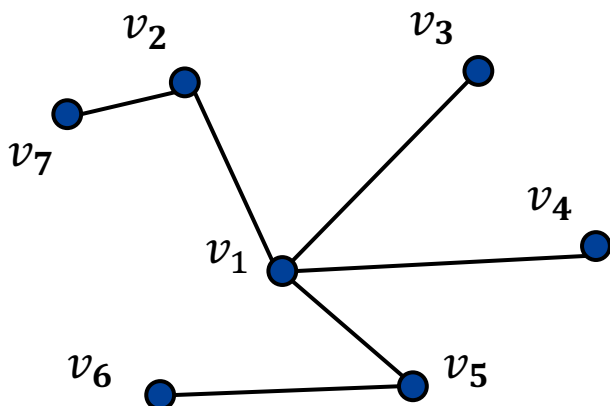
01.图论基本概念：树的定义

【定义】 无回路的连通无向图称为**无向树**，简称**树**。

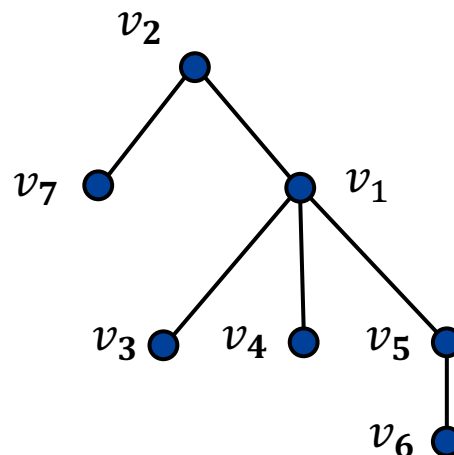
常用 T 表示树。树中最长路的长称为树的**高**。树的度为1的顶点称为**树叶**，其余的顶点称为**分枝点**。树的边称为**树枝**。

若无向图 G 至少有两个连通分支（最大可达的连通子图），每个都是树，则称 G 为**森林**，简称**林**。由定义，森林即每个连通分支都是树的图。

可以任意指定一个顶点为根来将一个无根树转化为**有根树**，而选择不同的根将产生不同的有根树



以 v_1 为根



以 v_2 为根

01.图论基本概念：生成树定义

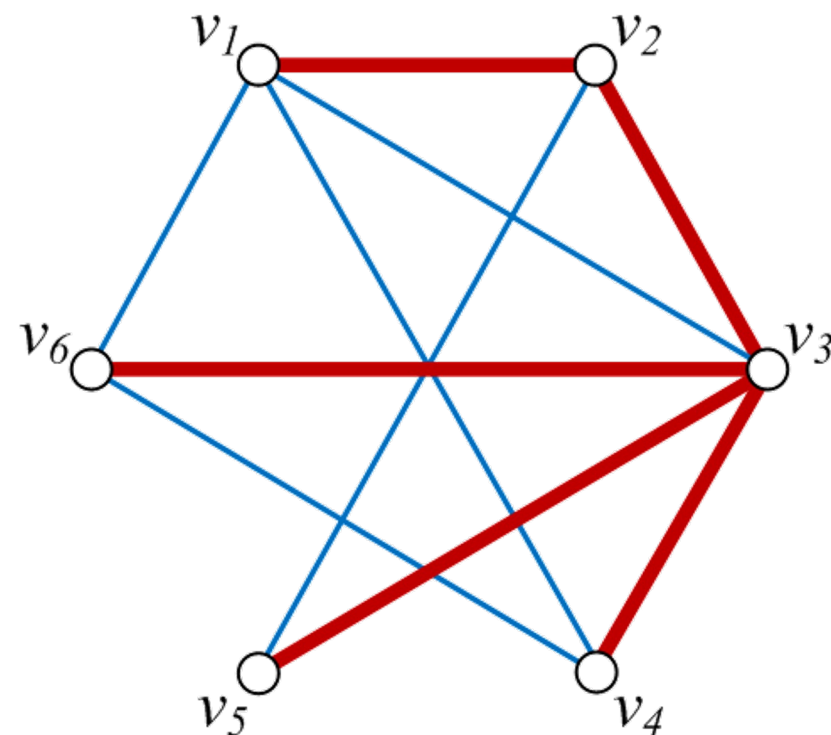
【定义】 设 T 是无向图 G 的子图并且为树，则称 T 为 G 的**树**。若 T 是 G 的树且为生成子图，则称 T 是 G 的**生成树 (spanning tree)**。

【例】 给出图 G 的生成树及其余树。

【解】 红色为生成树；蓝色为其余树。

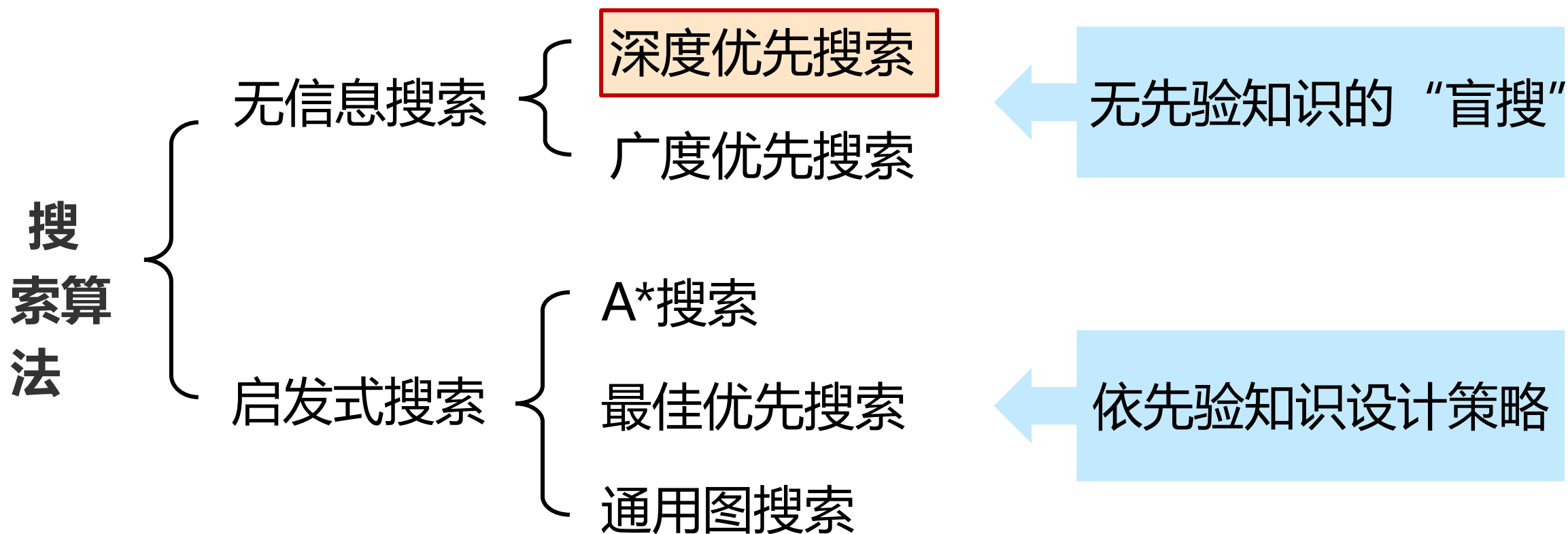
【注】 连通图的生成树不唯一。

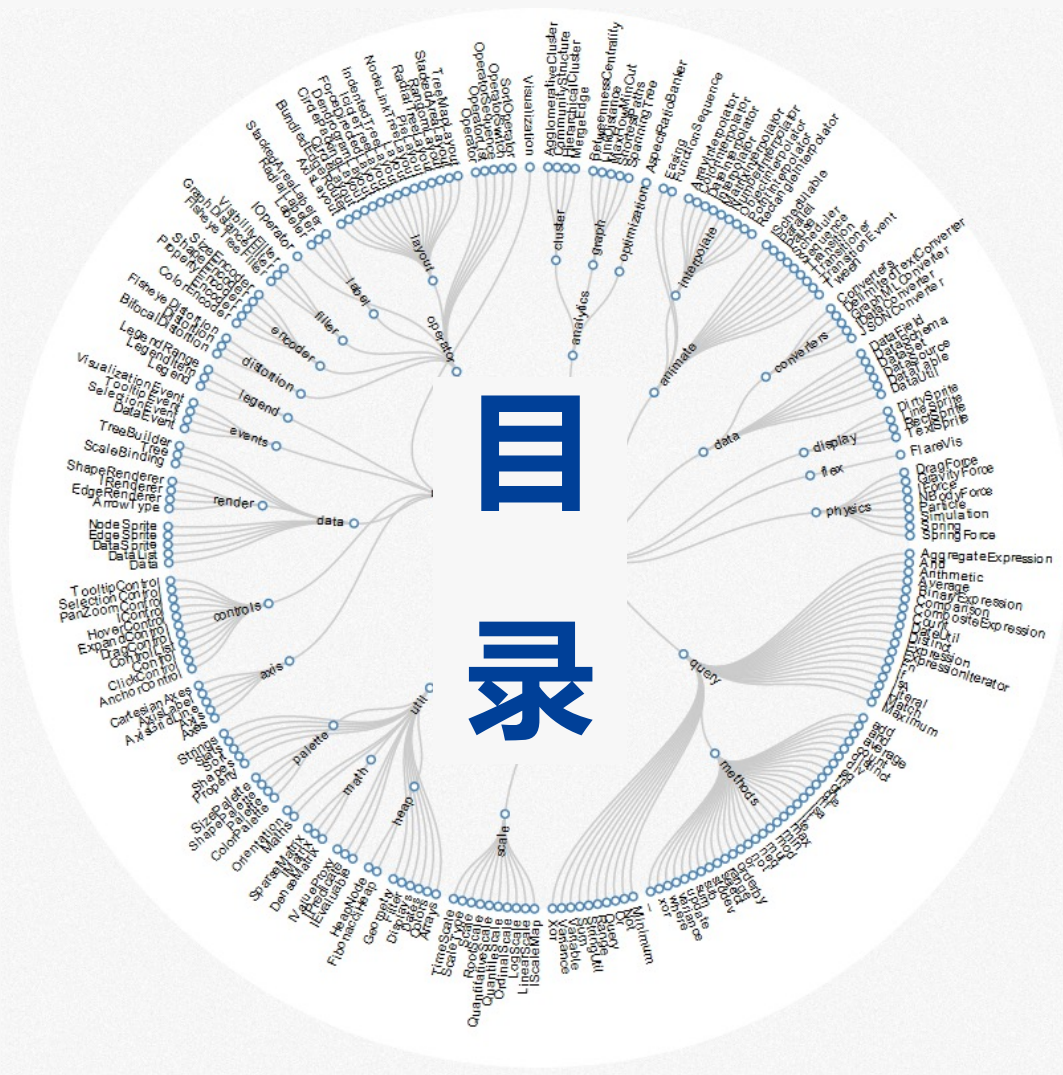
生成树以最少的边保持全图连通，是图的“骨架”。
很多工程问题（网络、供电、管网）追求在成本最小下确保连通——**生成树是第一性结构**。



搜索算法

定义：利用计算机按照某种方式穷举一个问题的部分或全部解空间，从而求出问题解的一种方法。





01

图论基本概念

02

深度优先搜索

03

广度优先搜索

深度优先搜索 (Depth-First Search, DFS)

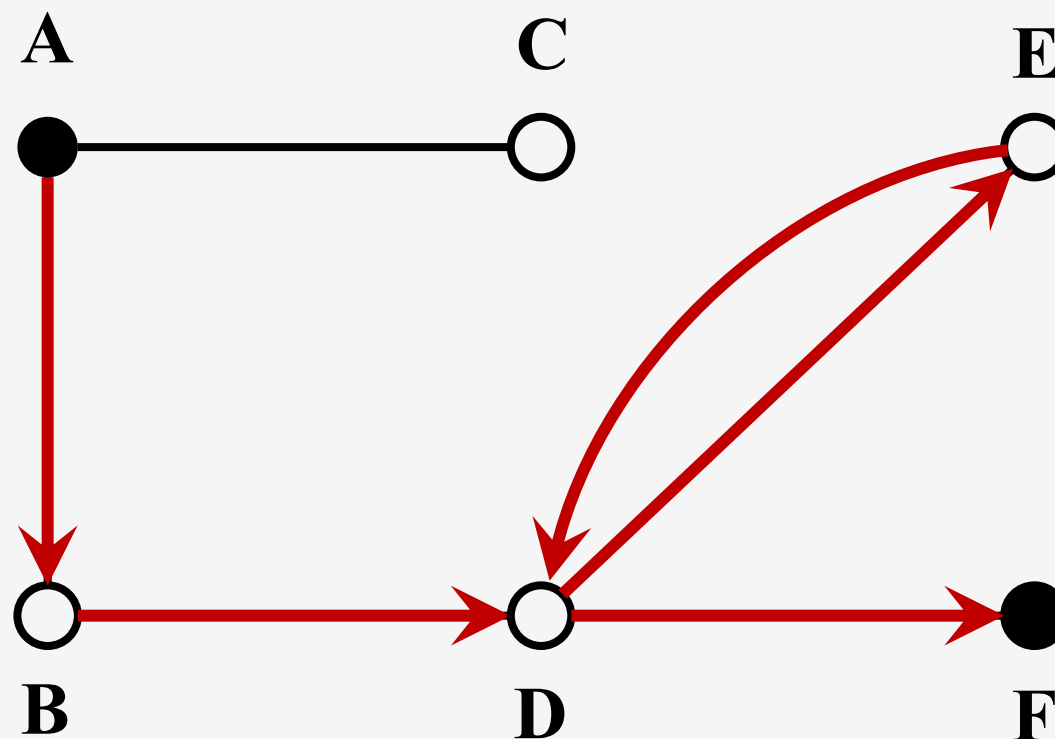
思路

- 从当前顶点出发，持续寻找与之相邻且未被访问过的顶点。
- 如果再找不到未访问过的顶点，则退回至前一个顶点重复该步骤。

【例】从A点出发，使用DFS搜索到F点的路径。

“不断深入，搜索回溯”

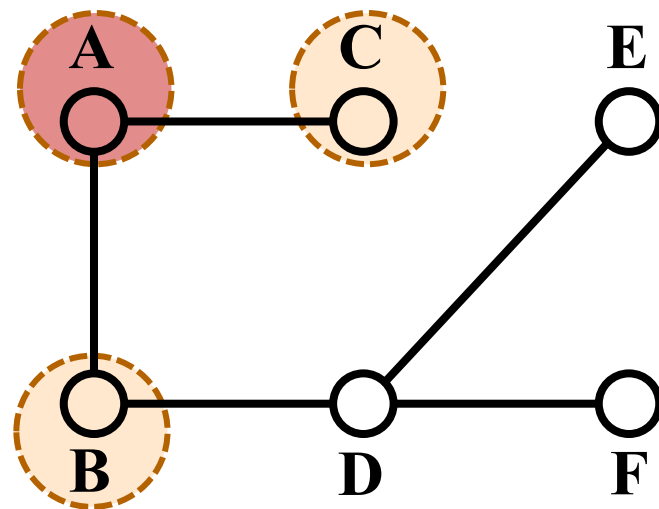
“不撞南墙不回头”



算法描述

初始：所有顶点都未访问过

DFS(G, v)	
Input: $G = (V, E)$, $v \in V$	
Output: $\text{visited}(u) = \text{true}$ for all $u \in V$ $\text{visited}(v) = \text{true}$	
1.	for each edge $(u, v) \in E$
2.	if $\text{visited}(u) = \text{false}$
3.	DFS(G, u)
4.	endif
5.	endfor

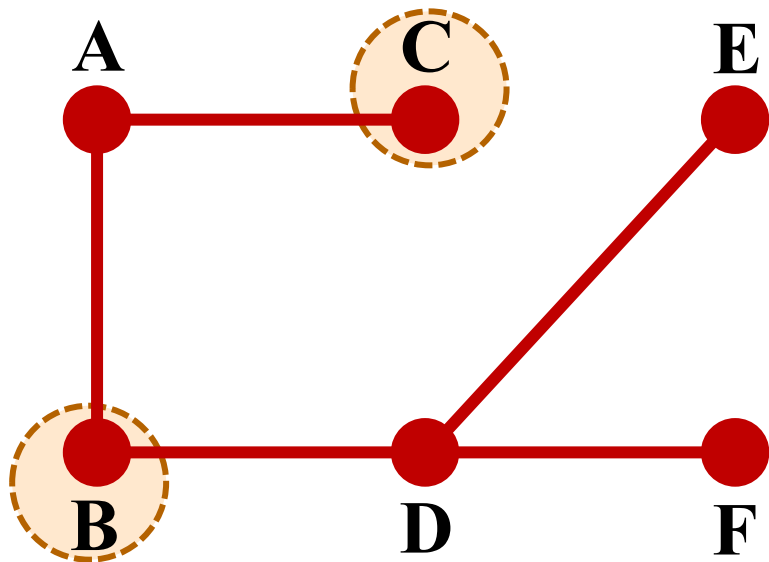


➤ 将节点 v 标记为“已访问”

➤ 考虑与节点 v 相邻的所有节点

➤ 如果有某个节点 u 之前未被访问过，那么调用DFS(G, u)函数

算法搜索示例



伪代码→

DFS(G, v) → **A**
1. **visited**(v) = true
2. **for** each edge $(u, v) \in E$
3. **if** **visited**(u) = false
4. DFS(G, u)
5. **endif**
6. **endfor**

访问点→	A	B	D	E	D	F	D	B	A	C	A
栈变换→											
				D		D					
			B	B	B	B					
	A	A	A	A	A	A			A		
	1	2	3	4	5	6	7	8	9	10	11

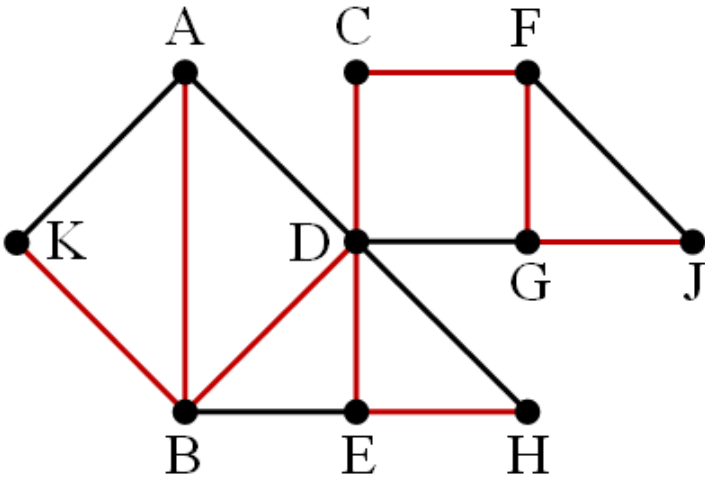
算法结束时遍历全图

DFS不能保证最短！

生成树 vs 深度优先搜索算法(DFS)

更多DFS访问结果：以A为起点，按照字母顺序访问结果如右图所示。

对DFS函数的访问入栈情况如下：



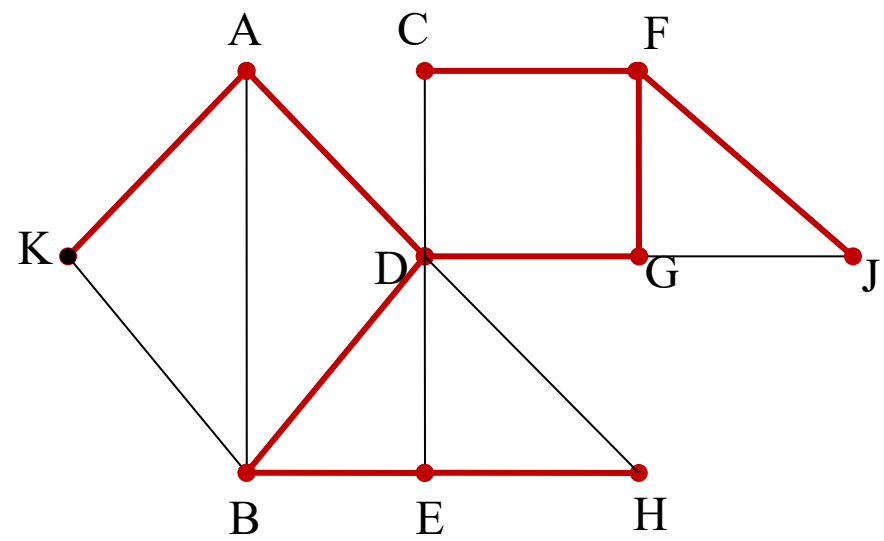
A	B	D	C	F	G	J	G	F	C	D	E	H	E	D	B	K	B	A
						G												
					F	F	F											
				C	C	C	C	C				E						
			D	D	D	D	D	D	D		D	D	D					
		B	B	B	B	B	B	B	B	B	B	B	B	B		B		
	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

生成树 vs 深度优先搜索算法(DFS)

以K为起点的访问顺序:

K, A, D, B, E, H, E, B, D, G, F, C, F, J, F, G, D, A, K

对DFS函数的访问入栈情况如下:

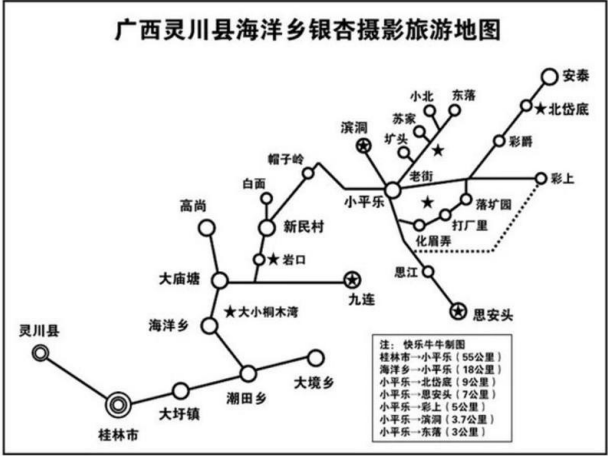


K	A	D	B	E	H	E	B	D	G	F	C	F	J	F	G	D	A	K
					E						F		F					
				B	B	B				G	G	G	G	G				
			D	D	D	D	D		D	D	D	D	D	D	D			
		A	A	A	A	A	A	A	A	A	A	A	A	A	A	A		
	K	K	K	K	K	K	K	K	K	K	K	K	K	K	K	K	K	

DFS应用：旅游参观



- A、演出：丛林的呼唤
- B、演出：杰克船长
- C、游乐：加勒比海盗
- D、游乐：爱丽丝梦游仙境
- E、游乐：小飞象
- F、演出：夜光幻影秀
- G、游乐：喷气背包飞行器
- H、游乐：太空幸会史迪奇
- I、游乐：巴斯光年星际营救
- J、游乐：创极速光轮
- K、游乐：弹簧狗团团转
- L、游乐：抱抱龙冲天战车
- M、游乐：胡迪牛仔嘉年华





图论基本概念

深度优先搜索

广度优先搜索

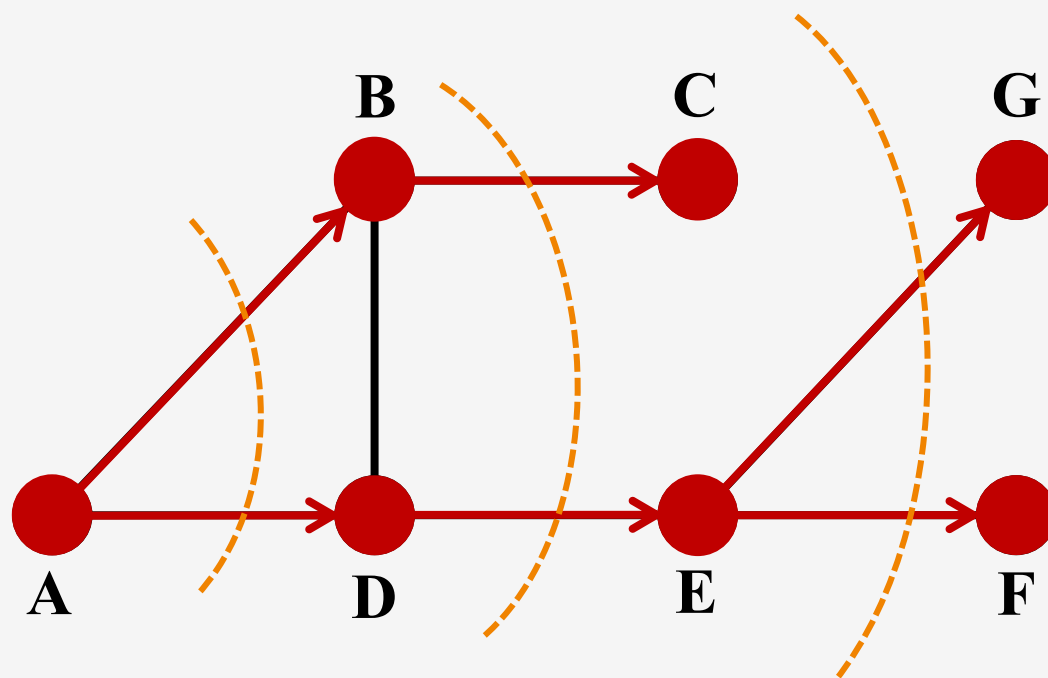
广度优先搜索 (Breadth-First Search, BFS)

【思路】 标记初始节点，并访问与之相邻且未被标记的节点。一旦找到，节点就被标记。迭代一直持续到所有的节点都被访问和标记。

【例】 从A点出发，使用BFS搜索到其他点的路径。

波状推进

不断扩大半径



初始：所有顶点都未访问和标记

BFS(G, s) $\rightarrow A$

Input: $G = (V, E), s \in V$

Output: For all vertices u reachable from s , $\text{dist}(u)$ is set to the distance from s to u

1. **for each** $u \in V$ **do**

2. $\text{dist}(u) = \infty$

3. $\text{dist}(s) = 0$

4. $Q = [s]$ (queue containing just s)

5. **while** Q is not empty **do**

6. $u = \text{eject}(Q)$

7. **for each** edge $(u, v) \in E$ **do**

8. **if** $\text{dist}(v) = \infty$

9. $\text{inject}(Q, v)$

10. $\text{dist}(v) = \text{dist}(u) + 1$

$\rightarrow$ 所有节点标记为 ∞

$\rightarrow$ 将节点 s 标记为0

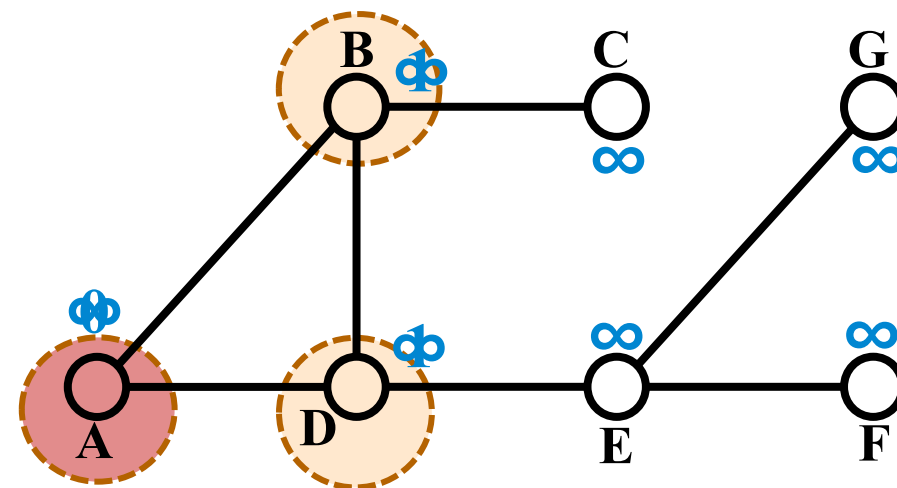
$\rightarrow$ 将节点 s 加入队列 Q

$\rightarrow$ 队列 Q 非空时

$\rightarrow$ 访问当前队列的第一个节点

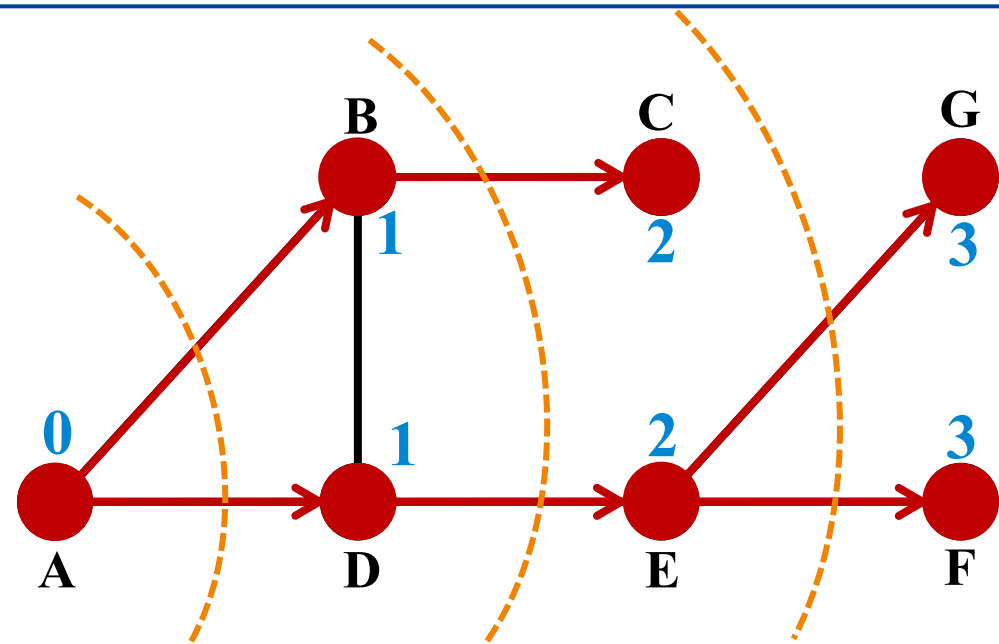
$\rightarrow$ 考虑与节点 u 相邻的所有节点

$\rightarrow$ 如果有某个节点 v 之前未被标记，那么将节点 v 加入队列，并标记为 $\text{dist}(u) + 1$



$Q = [B, D]$

算法搜索示例



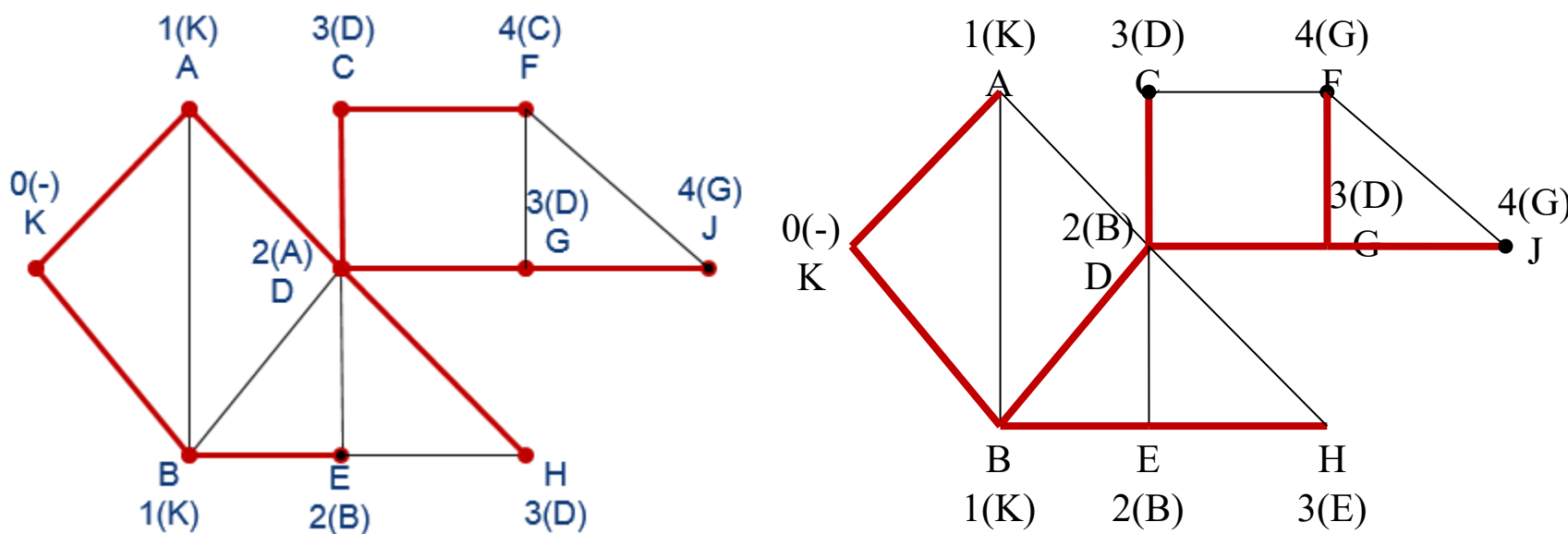
伪
代
码

```
BFS(G, A)
5. while Q is not empty do
6.   u = eject(Q)
7.   for each edge (u, v) ∈ E do
8.     if dist(v) = ∞
9.       inject(Q, v)
10.    dist(v) = dist(u) + 1
```

访问点	队列变换				
1	<table><tr><td>A</td><td></td><td></td><td></td></tr></table>	A			
A					
2	<table><tr><td>B</td><td>D</td><td></td><td></td></tr></table>	B	D		
B	D				
3	<table><tr><td>D</td><td>C</td><td></td><td></td></tr></table>	D	C		
D	C				
4	<table><tr><td>C</td><td>E</td><td></td><td></td></tr></table>	C	E		
C	E				
5	<table><tr><td>E</td><td></td><td></td><td></td></tr></table>	E			
E					
6	<table><tr><td>F</td><td>G</td><td></td><td></td></tr></table>	F	G		
F	G				
7	<table><tr><td>G</td><td></td><td></td><td></td></tr></table>	G			
G					
8	<table><tr><td></td><td></td><td></td><td></td></tr></table>				

生成树 vs 广度优先搜索算法(BFS)

在使用BFS算法时，如果未说明顺序（如字母序），有些地方边可任意选取。不同的选择导致不同的生成树。例如以K为起点，选择不同的顺序可以生成不同的生成树，如下两图所示。





巡逻机器人

技术

- 5G通信
- 物联网
- 云计算

功能:

- 环境感知
- 动态决策

工作:

- 基础性
- 重复性
- 危险性



图论基本概念

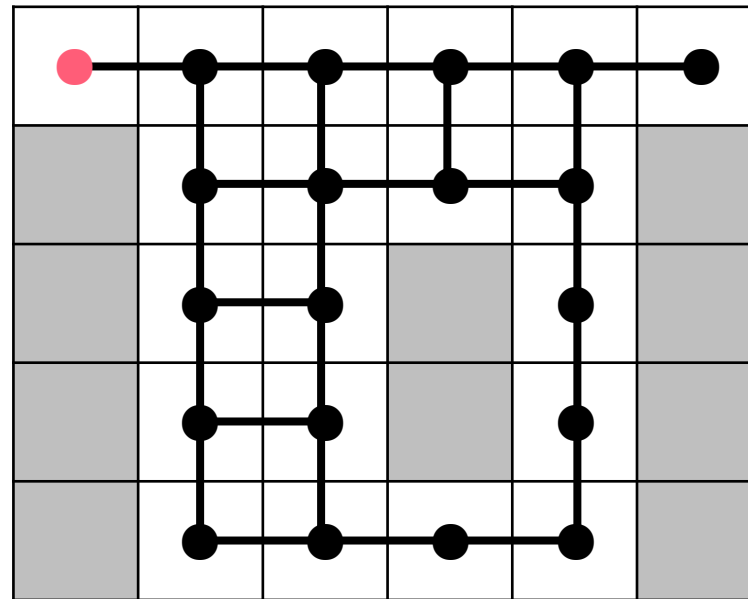
- 无向图 (定义、点、边、度)
- 有向图 (定义、出度、入度)
- 子图、树

深度优先搜索 (DFS)

- 基本算法
- 栈的应用

广度优先搜索 (BFS)

- 基本算法
- 队列应用



DFS(G, v)

1. $\text{visited}(v) = \text{true}$
2. **for** each edge $(u, v) \in E$
3. **if** $\text{visited}(u) = \text{false}$
4. DFS(G, u)
5. **endif**
6. **endfor**

谢谢!

